DuocUC | ONLINE



Guía de aprendizaje

Implementación del patrón arquitectónico con Backend for Frontend (BFF)

Exp 2 – Semana 5

Desarrollo Backend III (PBY2203)

Facilitador disciplinar: Ana Karina Villagrán Ibarra

Asesor par: Ignacio Villarroel

Índice

Introducción a la semana....3
Resultado de aprendizaje....4
Conceptos relevantes....4
Preguntas activadoras....5
Actividad....5
Estrategias de implementación....6
1. Crear Backends independientes por cada tipo de cliente....7
2. Diseño de Endpoints personalizados....9
3. Aprovechar microservicios y delegar funciones....11
Ejemplo práctico de refactoring de monolito....14
Lectura complementaria....18
Cierre de la semana....19
Referencias....20
Lecturas de la semana....20
Apuntes....21

Introducción a la semana



Esta semana te centrarás en explorar y aplicar el patrón arquitectónico Backend for Frontend (BFF), una estrategia ideal para mejorar la interacción entre diferentes frontends, como aplicaciones web y móviles y sus respectivos backends. A través de esta práctica, aprenderás a personalizar el backend para que responda de manera más efectiva a las necesidades específicas de cada interfaz de usuario.

En esta guía de aprendizaje encontrarás:

- ¿Qué estrategias de implementación existen?
- ¿Cómo crear Backend independientes por cada tipo de cliente?
- ¿Cómo diseñar Endpoints personalizados?
- ¿Cómo Aprovechar microservicios y delegar funciones?

Implementando el BFF, optimizarás la integración entre múltiples frontends y backends, configurando el servidor para asegurar una estructura escalable y modular. Esto te permitirá crear una base sólida y adaptable, ideal para responder a futuros cambios y mejorar la experiencia del usuario en cada plataforma.

Resultado de aprendizaje

El estudiante será capaz de:

RA2. Desarrolla e implementa patrones arquitectónicos conocidos, a partir del diseño, que den respuesta a diferentes frontends.

Indicador de logro:

IL4. Aplica el patrón arquitectónico Backend for Frontend (BFF) para la integración eficiente entre múltiples frontends y backends.

IL5. Configura el servidor utilizando el BFF para optimizar la interacción entre backends y frontends, asegurando la escalabilidad y modularidad del sistema.

Conceptos relevantes

Complejidad	Escalabilidad	Monolito
Patrón de diseño	API Gateway	Microservicio

Preguntas activadoras

- ¿Qué necesidades específicas crees que tienen los diferentes frontends de aplicaciones y cómo pueden afectar la estructura del backend?
- ¿Qué deberías considerar para implementar un patrón de BFF en un proyecto?
- ¿En qué casos específicos un patrón BFF puede no ser la mejor opción? ¿Qué alternativas existen y cuándo podrían ser más adecuadas?

Actividad

Descripción de la actividad

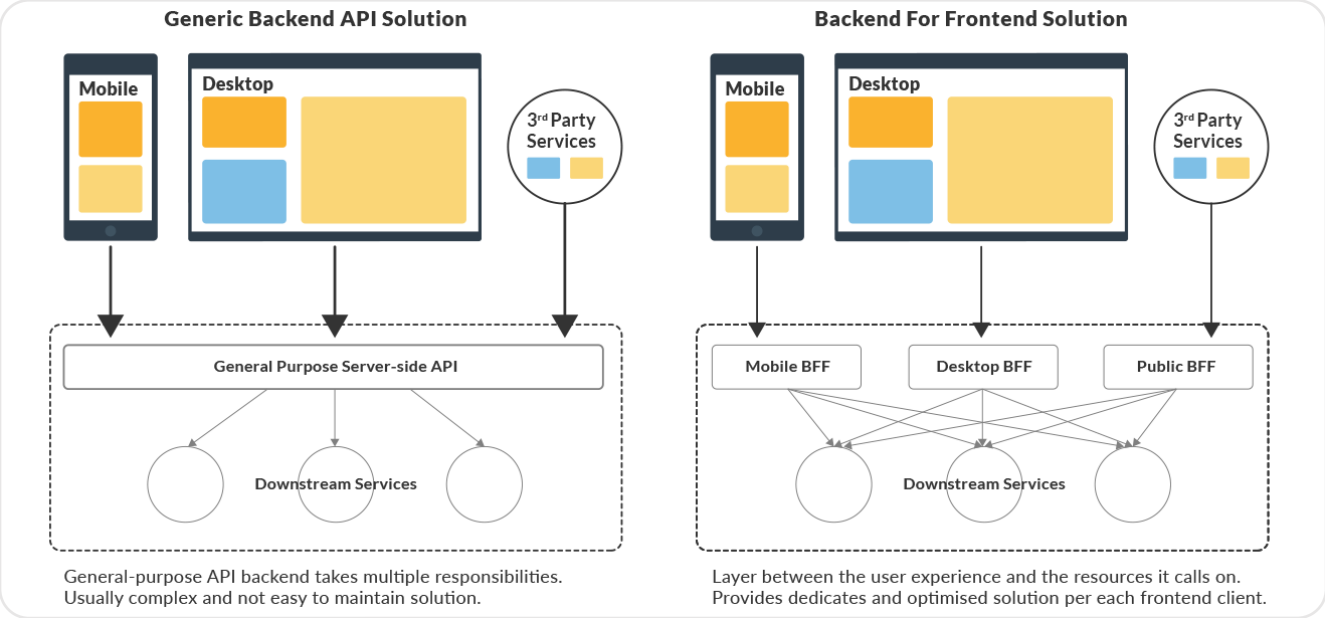
En esta quinta semana, realizarás una actividad sumativa individual llamada “Optimizando el patrón arquitectónico Backend for Frontend”, donde aplicarás el patrón Backend for Frontend (BFF) para optimizar la integración entre múltiples frontends (como aplicaciones web y móviles) y un backend centralizado o distribuido, según la estrategia que elijas. Implementarás BFF para garantizar que cada frontend acceda a los datos y funcionalidades necesarias de manera eficiente y personalizada, asegurando la escalabilidad y modularidad del sistema.

Estrategias de implementación

Como abordamos en la semana anterior, existen diferentes estrategias para la implementación del patrón arquitectónico Backend for Frontend (BFF) que dependen profundamente de diferentes factores a considerar. Estas estrategias varían en función de las necesidades específicas de la empresa, la arquitectura existente y los objetivos de cada equipo de desarrollo.

Figura 1

Comparación entre solución API Backend genérica y solución Backend for Frontend (BFF)



Nota. La imagen compara un enfoque de API backend genérico frente a una arquitectura BFF. Szczepanik, M. (2021). *Backend for frontend (BFF) pattern— why do you need to know it?* https://medium.com/mobilepeople/backend-for-frontend-pattern-why-you-need-to-know-it-46f94ce420b0

En este recurso, exploramos tres estrategias clave para implementar el BFF:

- **Crear backends independientes por tipo de cliente,** lo que permite una personalización completa para cada interfaz.
- **Diseñar endpoints personalizados,** una solución eficiente que adapta respuestas específicas para cada cliente.
- **Aprovechar arquitecturas de microservicios y delegar funciones,** que integra el BFF como una capa intermedia optimizadora.

Cada estrategia tiene ventajas y desafíos que deben analizarse cuidadosamente. Factores como el tamaño del equipo, la complejidad de los frontends, la personalización requerida y los costos asociados desempeñan un papel crucial en la selección de la estrategia adecuada. Por ello, a continuación profundizaremos en las estrategias mencionadas:

1. Crear Backends independientes por cada tipo de cliente

En esta estrategia deberás desarrollar un BFF específico para cada cliente (por ejemplo, web y móvil), asegurando que cada backend tenga lógica adaptada a las necesidades de su interfaz. Esto puede implicar mantener repositorios de código completamente independientes.

Los factores a considerar se deben plantear como preguntas, por ejemplo:

- ¿Necesito avanzar de manera independiente con equipos dedicados para cada tipo de cliente de la solución de software?
- Según el tipo de cliente, ¿poseo diferentes reglas de negocio y comportamientos en mi solución de software?
- Según el tipo de cliente, ¿tendré que considerar comportamientos de demanda, latencia y disponibilidad diferentes?

Otra forma de analizar si esta es la estrategia adecuada es ver sus ventajas y desventajas.

Tabla 1

Ventajas y desventajas de la estrategia 1

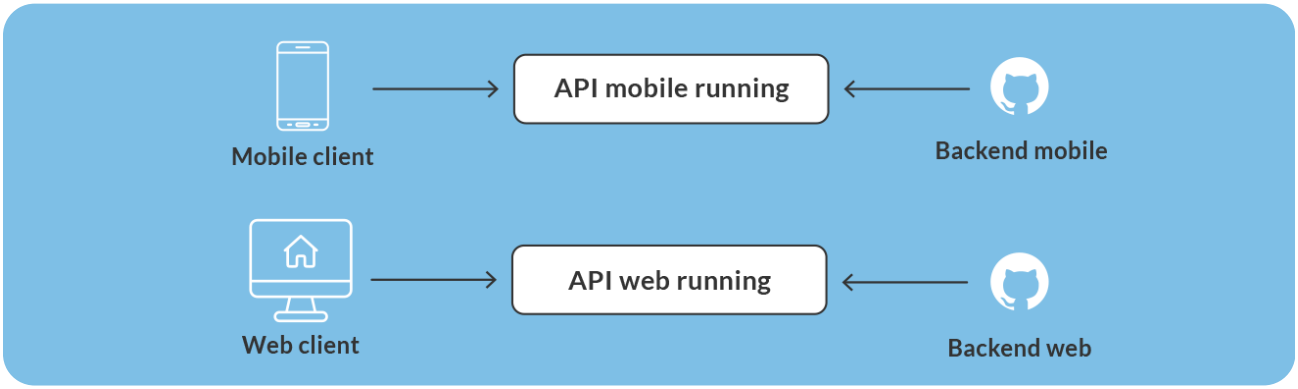
Ventajas	**Desventajas**
Personalización especifica	Duplicación de esfuerzos
Autonomía de equipos para cada cliente	Mayor complejidad en la arquitectura
Mejor mantenibilidad	Mayores costos de infraestructura
Escalabilidad individual en caso de ser necesario	Desafíos de seguridad y consistencia
Reducción de la complejidad de código en el frontend	Requiere mayor coordinación entre diferentes equipos
Más simple a la hora de pensar solo en un cliente especifico.	Mayor curva de aprendizaje y complejidad técnica
	Mayor esfuerzo de gestión de versiones y despliegues

¿Cómo se implementa?

En la siguiente imagen se muestra un diagrama de cómo implementar un BFF según el tipo de cliente:

Figura 2

BFF por tipo de cliente



Nota. La imagen muestra la implementación de un BFF con backends independientes entre ellos.

2. Diseño de Endpoints personalizados

Esta estrategia implica crear endpoints específicos para cada tipo de cliente, como por ejemplo /web/articles y /mobile/articles, lo cual permite que el sistema se adapte mejor a las necesidades de cada frontend.

Los factores a considerar podrían ser:

- ¿Cuántas personas están disponibles para desarrollar funcionalidades específicas para cada cliente?, ¿solo tengo un equipo disponible para todos los canales?
- Cuando los equipos de desarrollo son pequeños, ¿es importante la customización según el tipo de cliente?, ¿la personalización es una necesidad obligatoria?
- Antes de ejecutar mis lógicas de negocio, ¿necesito transformar la data que llega desde el cliente para darle una respuesta adecuada?

También revisa sus ventajas y desventajas para tomar la decisión:

Tabla 2

Ventajas y desventajas de la estrategia 2

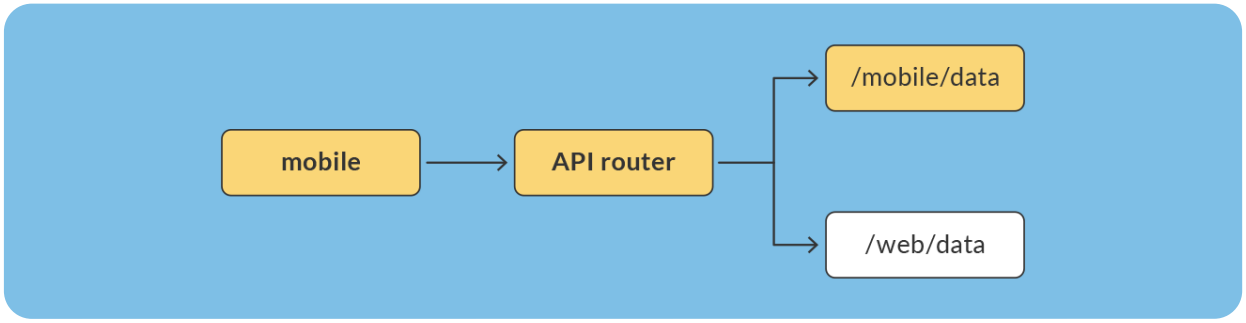
Ventajas	**Desventajas**
Optimización de la experiencia de usuario	Incremento en la complejidad del backend
Reducción de la carga de trabajo a las capas de negocio	Posibles duplicaciones de lógicas de negocio
Flexibilidad en la evolución de frontends	Dificultad en la gestión de versiones
Seguridad y control de acceso centralizado para todos los clientes	Riesgos de sobre-ingeniería
Mantenimiento más simple	
Un mismo equipo puede dar mantención a todos los clientes	

¿Cómo se implementa?

La siguiente imagen muestra un diagrama de cómo se implementa BFF usando endpoints personalizados.

Figura 3

BFF con endpoint personalizados



Nota. La imagen muestra la implementación de un BFF con endpoint personalizados independientes entre ellos. Si el cliente es Mobile, seguirá la ruta para el cliente Mobile.

3. Aprovechar microservicios y delegar funciones

En una arquitectura de microservicios, el BFF puede actuar como una capa intermediaria que combina respuestas de varios servicios antes de enviarlas al frontend.

Los factores a considerar podrían ser:

- ¿Ya posees una arquitectura de microservicios?
- ¿Existen lógicas de negocio condicionadas por el tipo de cliente?

También sugerimos revisar sus ventajas y desventajas para tomar la decisión:

Tabla 3

Ventajas y desventajas de la estrategia 3

Ventajas	**Desventajas**
Mejor personalización de respuestas	Aumentar la complejidad en el BFF
Escalabilidad independiente	Posible duplicación de lógica entre BFF y microservicios
Desacoplamiento de lógicas de negocio	Costos adicionales de infraestructura
Reducción de cargas en el frontend	Riesgos de latencia
Flexibilidad para implementar nuevas funcionalidades	Desafíos de seguridad

Consideraciones generales

Con el análisis de las distintas estrategias mencionadas, siempre hay que destacar aquellos factores comunes en los que deberás meditar para tomar una mejor decisión.

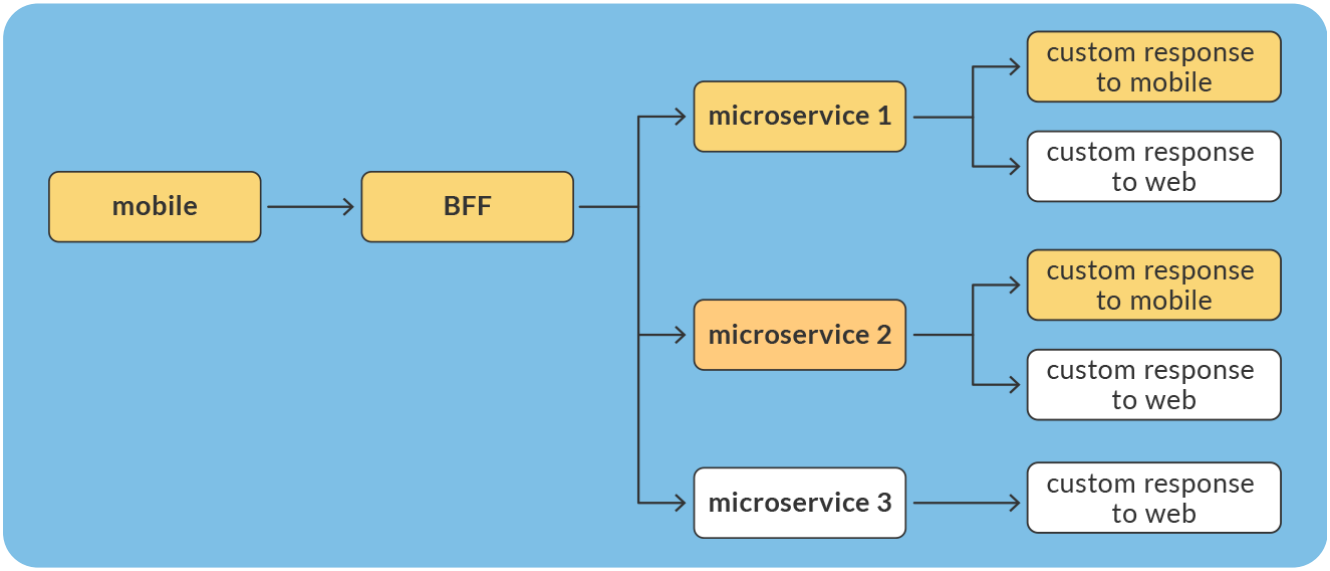
- **Complejidad de los frontends:** evalúa cuán distintos son los requisitos de cada frontend (web, móvil, IoT, etc.) y qué nivel de personalización requieren.
- **Tamaño del equipo:** considera la cantidad de personal para realizar mantenciones y actualizaciones del software.

¿Cómo se implementa?

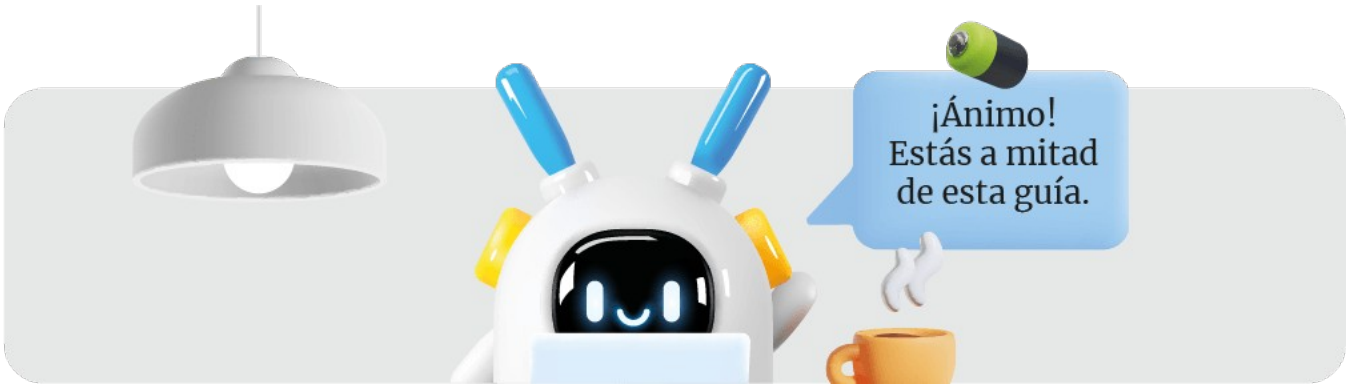
La siguiente imagen muestra un diagrama de cómo un BFF podría delegar capacidades y respuestas personalizadas según el cliente:

Figura 4

BFF con delegación a microservicios



Nota. La imagen muestra la implementación de un BFF con delegación a microservicios.



Ejemplo práctico de refactoring de monolito

Para ver una implementación real, analicemos el caso del repositorio monolítico que lista artículos.

Si bien el actual repositorio no cuenta con ninguna estrategia de BFF implementada, este nos ayudará a cumplir el objetivo de refactorizar el proyecto, puesto que el refactoring no solo es un medio para mejorar el código, sino también un **facilitador clave para implementar estrategias de BFF.**

Links de Interés

Ejemplo de monolito: https://ava.duoc.cl/bbcswebdav/xid-5106124_1

Considera los siguientes antecedentes:

- Cuentas con un equipo de implementación pequeño.
- Existe la necesidad de optimización específica para un cliente web y un cliente móvil.
- Debes considerar la mantenibilidad a la hora de extender la solución a nuevos tipos de clientes.
- El tiempo es un factor crítico, y el objetivo se requiere cumplir en el menor tiempo posible.

Análisis de cada estrategia:

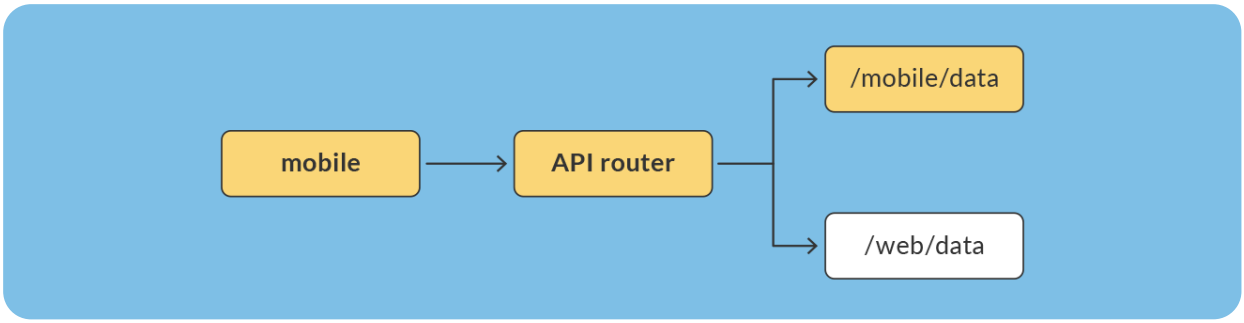
Debes definir qué estrategia utilizar, para ello analicemos las estrategias disponibles:

- **Backends independientes:** el equipo, siendo pequeño, podría verse limitado para mantener múltiples repositorios con capacidades similares pero personalizadas para cada cliente. Además, se ha identificado la intención de utilizar un enfoque monolítico para adaptarlo a las necesidades específicas de los clientes actuales (web y móvil).
- **Endpoints personalizados:** esta estrategia ofrece una implementación más sencilla y rápida, manteniendo el código centralizado, lo que facilita las mantenciones y permite al equipo reutilizar capacidades previamente desarrolladas en el mismo proyecto.
- **Delegación con microservicios:** la estrategia de microservicios no aplica, ya que no es posible, puesto que no existen aún.

Para este caso, se decide la estrategia de **Endpoints personalizados**.

Figura 5

Endpoint por tipo de cliente



Nota. La imagen muestra la implementación de un BFF que crea diferentes paths en su API dependiendo del cliente que consulta.

La siguiente imagen muestra un ejemplo de cómo se podría refactorizar el proyecto monolito conservando su lenguaje de programación y solo disponibilizando diferentes canales para cada tipo de cliente. En tu propuesta de solución (actividad sumativa) deberás abordar la refactorización de servicios considerando Java Spring Boot.

Figura 6

Ejemplo de refactorización



Nota. La imagen muestra un ejemplo de refactorización para que el monolito pueda considerar rutas diferentes según el cliente. Cada cliente será responsable de consultar el path que le corresponde. Microsoft (2006). Power Shell. (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

Figura 7

Método backend a refactorizar

```
async function getData(req, res) {
    try {
        const clientType = req.query.client || (req.headers['user-agent'].includes('Mo
        const data = await dataService.getData();

        // Personalizar respuesta según el cliente
        let transformedData;
        if (clientType === 'web') {
            transformedData = data.map(item => ({
                id: item.id,
                title: item.title,
                content: item.content,
                image: item.imageUrl
            })).slice(0, 150); // Contenido completo para web
        } else {
            transformedData = data.map(item => ({
                id: item.id,
                title: item.title,
                summary: item.content,
                image: item.thumbnailUrl // Imagen pequeña para móviles
            })).slice(0, 50); // Contenido resumido para mobile
        }

        res.json(transformedData);
    } catch (error) {
        res.status(500).json({ error: 'Error fetching data' });
    }
}
```

Nota. La imagen muestra la implementación original del código. Para obtener el resultado de la Figura 2, necesitamos separar la función getData() en dos métodos.

La idea de dividir el método **getData**() (Figura 4) en dos métodos independientes es permitir la invocación de dos rutas completamente separadas (Figura 3). Este enfoque reducirá la complejidad del código, ya que al eliminar el uso del “if” que determina si se trata de un cliente web o móvil, el código no necesitará bifurcarse en función de una variable. Esto se relaciona con la complejidad algorítmica: cada "if" que se elimina disminuye esta métrica, lo que resulta en un código más simple y mantenible.

Lo que queda, es entregarle a cada cliente su ruta personalizada.

Lectura complementaria

Puedes encontrar más información sobre Backend for Frontend en el siguiente artículo:

https://aws.amazon.com/es/blogs/mobile/backends-for-frontends-pattern/

Cierre de la semana



A lo largo de estos días, exploraste cómo el BFF optimiza la interacción entre distintos frontends (como aplicaciones web y móviles) y el backend, adaptando las respuestas a las necesidades específicas de cada interfaz de usuario.

Esta semana, te enfocaste en implementar el BFF para crear una arquitectura escalable y modular. A través de diversas estrategias de implementación, como la creación de backends independientes, el diseño de endpoints personalizados y la delegación de funciones en una arquitectura de microservicios, adquiriste herramientas para tomar decisiones estratégicas y efectivas en función de los requisitos y limitaciones de cada proyecto.

Analizaste las ventajas y desventajas de cada enfoque, comprendiendo que factores como el tamaño del equipo, la complejidad de los frontends y la personalización de las respuestas son cruciales para elegir la estrategia adecuada.

Ahora, tienes una base sólida para diseñar sistemas más eficientes y adaptados a los distintos dispositivos y tipos de usuarios, contribuyendo a una experiencia más optimizada.

Referencias

- Caules, A. (2021). *¿Qué es el patron BFF?* https://www.arquitecturajava.com/que-es-el-patron-bff/
- Newman, S. (2015). Pattern: Backends For Frontends. https://samnewman.io/patterns/architectural/bff/

Lecturas de la semana

- AWS. (2024). *Integración de API – Backend para frontend.* https://docs.aws.amazon.com/es_es/prescriptive-guidance/latest/micro-frontends-aws/api-integration-data-fetching.html
- Newman, S. (2015). *Pattern: Backends For Frontends.* https://samnewman.io/patterns/architectural/bff/

Apuntes



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.